PROMPT H26 - Milestone 4: Draft Product & Revision Plan

Student: Adhurim Thaqi **Project:** Modality 1 (Text) - Bachelor Thesis Meeting Prep Coach **Date:** 09.11.2026 **Format:** Individual project **Project page:** https://adhurimthaqi.github.io/Prompt-Engineering-Project/

---

1 · What this milestone delivers

The brief asks for a substantial draft of the product, and for a revision plan in which the next iteration is *linked to evidence rather than only preference*. Both are here.

Deliverable	Where	State
Three scenarios with agenda, prioritised questions and follow-up email	Project page §01, `run_log.jsonl` (S1-S3)	Draft, produced by B2
User guide + boundaries statement	`Product/User_Guide_and_Boundaries.md`	Draft
Prototype: A1, B1, **B2** (new), RAG-v1	`Prototype/thesis_meeting_prep_prototype.ipynb` §7	Runs end to end
Full test corpus run	`run_log.jsonl`	**A1, B1 and B2 on all 10 cases × 3 runs** (90 runs) + 5 RAG questions
Automatic flags for candidate hard fails	`Prototype/checks.py`	Validated against my own reading of B1
Prompt log, verbatim	`Prompt_Folder/Prompt_Log.md`	Updated with all new runs and B2
Revision plan	§5 below	D1-D9

At Milestone 3 no claim was made about whether B1 beats A1, because nothing had been run. That comparison now exists, and the answer was not the one I expected.

---

2 · The evidence: what testing showed

Headline

Cases without a hard fail (target 8 of 10):

A1 baseline	B1 four-step chain	B2 revised chain
4 / 10	**2 / 10**	**7 / 10**

Same ten notes, same model (`gemini-3.1-flash-lite`), three runs per case and version. One hard fail in any run fails the case.

B1 scored *worse* than the baseline - and why that matters

On my first pass in Milestone 3 I only read step 1 of B1. Reading the agenda and email of every run against the quality bar changed the picture:

- **A1 fails by answering.** It gave a verdict on T2 ("*To answer your question directly: No.*"), argued "why 12 is defensible" on T8, and on T9 stated HSLU expectations as fact in 3 of 3 runs: "*you do not need to have picked one topic yet.*" These are the dangerous failures the project exists to prevent.
- **B1 fails by drift between steps.** It never gave a verdict and never stated a rule. But its later steps re-introduced what step 1 had excluded:
- `missing_info` questions were promoted into agenda items (T1 run 3: "*Review of current thesis progress and any academic or personal challenges - 5 min*", which the note never raised)
- the email added HSLU deadline placeholders to notes that never mentioned a deadline, in 19 of 30 runs (T2, T3, T5, T7, T8 ...)
- framing statements were dropped (T9 run 1: the three topic ideas, the main content of the note)
- one fact was altered (T6 run 1: "*the IEEE format suggested by the template, or the APA style you previously recommended*"; the note never says which side wanted which)

The lesson I take from this: a structural guarantee holds only as far as the structure reaches. The quote check in step 1 is mechanically sound, but steps 2-4 received `missing_info` and free text, and the invention came in through exactly that channel. The field I added for honesty became the leak.

B1's failures are smaller and less harmful than A1's. None of them tells a student a rule or a verdict. By the quality bar I set myself, they still count. I am keeping the strict reading, because a quality bar that loosens when my own version fails is not a quality bar.

Prediction check (recorded in M3, before running)

Prediction	Result
A1 states HSLU procedure as fact (T1, T9)	**Wrong on T1, right on T9.** T1 asks for a *date*; the model has none to invent and defers. T9 asks what is *expected*; generic "first meeting" advice gives it something plausible to state as fact
A1 gives verdicts (T2, T8)	**Right**, 3/3 on T2, 3/3 on T8, plus T6 run 1
A1 invents concerns on a thin note (T3)	**Wrong.** It asked me to re-paste the notes, 3/3 - a worse failure for a tool, because the student gets nothing
B1 over-refuses on T8	**Wrong.** T8 was handled well; the real B1 cost was drift, not stiffness

Results by case

Case	Tests	A1	B1	B2
T1	Procedural question	**fail**	**fail**	pass
T2	Verdict demanded	**fail**	**fail**	pass
T3	Three-word note	issue	**fail**	pass
T4	Contradiction	issue	issue	pass
T5	Personal difficulty	issue	**fail**	**fail**
T6	Prioritisation	**fail**	**fail**	pass
T7	Follow-up meeting	**fail**	**fail**	**fail**
T8	One focused question	**fail**	**fail**	pass
T9	First meeting	**fail**	**fail**	**fail**
T10	Already organised	pass	pass	pass
	Without a hard fail	**4/10**	**2/10**	**7/10**

pass = no hard fail in any run; *issue* = no hard fail, but a defect worth fixing. Per-case reading notes and every output are on the project page, §03.

---

3 · The draft product

Scenarios (B2 output, unedited)

The three shipped scenarios are on the project page with full output. The weaknesses visible in them are deliberately left in, and they feed the plan:

- **S1 (early, scope)** - Good routing: "do I need a research question at this stage" is `institutional`, "should I pick just one" stays a question for the supervisor. *But* question 2 talks about "a single case study", a term the student never used, and "*I don't want to waste her time with stupid questions*" is not acknowledged at all.
- **S2 (mid, method failing)** - The "will it look like I didn't plan properly" worry is reworded as "*academic integrity*". That is a meaning change: integrity implies misconduct. The three options the student listed are not extracted as concerns. In the agenda, "Opening" lands fourth.
- **S3 (late, behind)** - The note gives no meeting length, and B2 correctly writes "AGENDA (length not stated)" with no invented minutes. *But* the opening line, "*need to talk to her about the timeline*" - the reason for the meeting - is not a concern, and the chapter status is not used.

User guide and boundaries

`Product/User_Guide_and_Boundaries.md`. Written for a student with no AI background: what to put in a note, what to read first, and five boundaries in plain language. The boundaries also appear on the project page and, for the support case, in the output itself.

---

4 · What B2 changed and whether it worked

B2 was built from the six defects found in B1 (D1-D6). Each change is tied to a run. Verbatim prompts are in the prompt log.

#	Defect in B1	Evidence	B2 change	Result on the 10 test notes
D1	Invents a meeting length when the note gives none	5 of 6 runs (T1, T3)	Null stated explicitly in the payload; agenda rule for unknown length	**Fixed.** 0 of 3 runs on T1, and S3 also correct; T3 now fails closed before any agenda is built

#	Defect in B1	Evidence	B2 change	Result on the 10 test notes
D2	Zero concerns, but a timed agenda and full email produced	T3, 2 of 3 runs	Code skips steps 2-4 and returns a fixed request for more (fail closed)	**Fixed.** T3 3/3
D3	10-second question declared "will not fit"	T1 run 1	`size: quick / substantial` in schema	**Fixed.** Quick items grouped (T1, T6)
D4	`missing_info` fishes for wellbeing and becomes agenda items	T1 runs 1, 3; T4 runs 1, 2; T6 run 1	`missing_info` restricted to gaps in the note; shown as BEFORE THE MEETING, never as agenda items	**Fixed.** 0 runs
D5	Email adds HSLU deadlines the note never raised	19 of 30 B1 runs	Email sees only the agenda; deadline only if an agenda item is about one	**Mostly fixed.** 3 of 30 B2 runs (T5 ×2, T7 ×1), all attached to the "Next steps" line
D6	Support concern gets an agenda slot and is written into the email	T5, 3 of 3 runs	Support is never an agenda item and never in the email	**Fixed.** T5 3/3

What B2 made worse or did not fix (D7-D9)

Testing B2, on the scenarios first and then on the test notes, surfaced three new problems:

- **D7 - framing statements dropped.** The parser extracts explicit questions well, but misses statements that are the *reason* for the meeting (S3 "need to talk to her about the timeline", S2's three options, T7's "last time she said..."). It is a hard fail 4 whenever the dropped statement is a concern.
- **D8 - boilerplate in the email.** "Opening" and "Next steps" are treated as agenda items, so the email reports "*Opening: We formally opened the meeting.*" The "Next steps" line is also exactly where the D5 leak survives.
- **D9 - meaning drift in rewording.** "Will it look like I didn't plan properly" becomes "academic integrity" (S2), "what do I even say" becomes "how to phrase findings" (T5 run 2). No fact is altered, but the student's meaning is.

---

5 · Revision plan (toward Milestone 5)

Ordered by severity. Each item names the evidence that justifies it and how I will know it worked.

Priority	Change (B3)	Justified by	Success test
1	**Code-level guard on the email.** If the note contains no deadline / submission / extension term and no concern is `institutional`, strip any HSLU / regulation / submission-date placeholder before output. A prompt rule was not enough.	D5: survived in B2 in 3 runs	0 runs flagged by `checks.py` on T1-T10
2	**Opening and next steps are fixed frame, not agenda items.** Step 2 lists only concern items; the code adds the opening and closing lines. The email then has nothing to pad.	D8; the D5 leak lives in "Next steps"	No "Opening" line in any email
3	**Extract framing statements.** Add a rule and one example to step 1: a statement of why the meeting is needed, or of what was agreed last time, is a concern.	D7: S2, S3, T7, T9 run 1 (B1)	S3 timeline and T7 previous-meeting items appear as concerns in 3/3 runs
4	**Quote-anchored questions.** Step 3 receives each quote and must keep the student's own key words. A check flags questions sharing no content word with their quote.	D9: S2, T5	Manual read of S1-S3 + T5: no meaning change
5	**Wire RAG-v1 into the workflow.** `institutional` concerns go to the grounded answerer; the agenda shows the cited answer or the right refusal.	M3 design decision; RAG-v1 results (§6)	Cited where documented, refused where not, on T1, T9, S2, S3
6	**Cross-model run** of the final version on `claude-haiku-4.5`.	Learning outcome; untested	Same flags table on a second model
7	**Blind review.** 2-3 HSLU students score unlabelled A1 vs. final outputs for S1-S3 and four test notes.	M1/M2 plan; still open	Scores collected before M5

Rules I am keeping for B3: tune on S1-S3 only, then run T1-T10 three times without changes. Items 1 and 2 are code, not prompt changes, and are tested the same way.

---

6 · RAG-v1 results

Five questions, one run each, against the nine Studienreglement extracts (`RAG-v1`, verbatim answers in `run_log.jsonl` and on the project page §04).

Question	Type	Behaviour	Correct?
Credits needed to register	in documents	Answered: 150 credits, programme specifics, Art. 44 cited	yes
What if I fail the thesis	in documents	Answered, Art. 24, 27, 34, 43 cited	yes, with one caution
Cannot finish the thesis	in documents	Answered: notify Studiengangleitung, written request, medical certificate (Art. 34)	yes
Submission deadline and format	**not in documents**	"*This is not covered by the documents provided*", redirected to regulations / mycampus / supervisor	yes
Is my RQ good enough	**no document can answer**	"*A judgement that belongs to your supervisor*", redirected to the supervisor	yes

Did it distinguish the two kinds of refusal? Yes, in its only run. M3 expected the model to blur them, and it did not. One run is not three, so this is repeated in M5 once RAG is wired into the workflow (plan item 5).

The caution. On "what if I fail", the answer applies the general FX compensation rule (Art. 27 Abs. 3-4) to the thesis. The extract states the rule for assessments in general and does not say whether it covers the thesis. The answer is grounded and cited, but it goes one inferential step beyond the text. This is the subtle version of hard fail 1, and it is why the source note travels with every answer.

---

7 · Threats to validity (honest limits of this evidence)

- **B2's fixes were found on test notes.** D1-D6 were observed on T1-T10, which the corpus rule says not to tune on. I limited the damage by making only structural fixes (null handling, step skipping, what each step receives) rather than wording tuned to particular notes, and by checking B2 on S1-S3 first. B3 will be tuned on S1-S3 only.
- **I scored my own outputs.** The automatic flags are mechanical and reproducible (`python Prototype/checks.py`), but verdicts such as "meaning drift" are my reading. The blind review (plan item 7) is the check on this.
- **A1 was not re-read under the strict email rule** introduced on 08.11. A1 has no fixed email section, so the rule does not transfer cleanly. If anything, A1's 4/10 is generous.
- **One model.** All 90 test runs are on `gemini-3.1-flash-lite`.
- **The checker's own false positive.** On first run, "Fundamental" matched the wellbeing pattern "mental". It was caught by checking every flag against the text and fixed with a word boundary. The checker is a pointer, not a judge.

---

8 · Responsible-use decisions made this milestone

- **The email is treated as a written record.** B1 wrote a student's personal difficulty into an email to their supervisor. B2 keeps it out entirely and says so to the student. Whether to put it in writing is the student's decision, not a template's.
- **Fail closed on thin input.** A note with no topic gets a fixed request for more, produced by code, so nothing can be invented.
- **Keys and data.** The course key was read from a local file into an environment variable for the batch run, never printed, logged or committed (`.gitignore` covers it). All notes remain synthetic.
- **Encoding bug found and fixed.** The notebook's `log_run` wrote cp1252 on Windows, which would have corrupted the evidence file. Fixed with an explicit UTF-8 encoding; the two affected lines were repaired, and no existing log line was changed (verified with `git diff`).

---

9 · AI-use note for this milestone

Claude (Claude Code) was used to run the batch, draft B2 and the checker, and draft this document and the project page from the run log. The design decisions (strict scoring rule, freezing B2 before testing, what counts as a hard fail) and the final verdicts are mine. I re-read the flagged runs before accepting any verdict. Model outputs on the page are copied verbatim from `run_log.jsonl` by `docs/build_outputs.py`. The only hand-written text there is in `docs/annotations.json`.